



Projeto 1: Melhor Preço

Relatório de Implementação - Estrutura de Dados

1 Objetivo

O presente projeto tem como objetivo a construção de um buscador e comparador de preços para supermercados. Utilizando uma base técnica fornecida para o consumo de APIs externas, o foco do desenvolvimento centrou-se na implementação de estruturas de dados eficientes para o armazenamento de itens, filtragem de marcas e termos, e a ordenação final de orçamentos para determinar a melhor opção de compra para o usuário.

2 Arquitetura e Diagrama de Classes

O sistema foi modularizado garantindo a responsabilidade única de cada classe. A comunicação central ocorre a partir da classe `Main`, que orquestra a leitura de dados, instancia os `Supermercados` e processa as respostas.

- **ItemLista:** Responsável por armazenar os critérios de busca do usuário e aplicar regras de negócio (normalização de *strings*) para aceitar ou recusar produtos.
- **Orcamento:** Encapsula o resultado de uma busca em um supermercado específico, implementando `Comparable` para ditar as regras do ranqueamento.
- **ListaSequencial:** Estrutura de dados basilar que armazena todas as coleções de objetos do sistema.

3 Estruturas de Dados Desenvolvidas

- **Lista Sequencial (Desenvolvimento em Sala):** A classe `ListaSequencial<T>` é a estrutura central do projeto, desenvolvida e refinada em ambiente de sala de aula. Ela implementa o redimensionamento automático (`expande()`) e fornece um `Iterator` próprio.
- **Ordenação com Merge Sort:** Integrado à `ListaSequencial`, o algoritmo *Merge Sort* (método `ordenaMescla`) foi implementado para organizar os resultados finais com complexidade $O(n \log n)$.

4 Manual de Implantação e Uso

Para garantir a reprodutibilidade do projeto, seguem as instruções de compilação (*build*) e execução do programa.

4.1 Pré-requisitos

- Java Development Kit (JDK) 11 ou superior instalado[cite: 12].
- Gradle.

4.2 Configuração da Cesta (*Input*)

Antes de executar, o usuário deve criar ou editar um arquivo de texto plano chamado `lista-compras.txt` na raiz do projeto. O formato aceito por linha é delimitado por ponto e vírgula (;):

Termo de Busca; Termos Obrigatórios (separados por vírgula); Marcas Aceitáveis

Exemplo de arquivo de entrada:

```
arroz; 5kg; urbano, tio joao  
leite; integral; tirol
```

4.3 Compilação e Execução (*Build e Run*)

Sendo este um projeto gerido via **Gradle**, é imprescindível que o ambiente possua o **JDK** instalado, sendo recomendada a utilização do **Java 25**. Recomenda-se a execução através de uma IDE (como IntelliJ IDEA ou Eclipse) que suporte projetos com dependências do Lombok e org.json nativamente.

1. Importe o projeto Gradle para a IDE.
2. Certifique-se de que o arquivo `lista-compras.txt` encontra-se na raiz do diretório de execução.
3. Sincronize as dependências do Gradle.
4. Localize a classe `Main.java` e execute o método `main`.

4.4 Demonstração de Uso e Saída Esperada

Ao executar o programa com a internet conectada, o sistema consumirá as APIs e imprimirá diretamente no console o ranqueamento ordenado seguido pelo detalhamento do vencedor:

```
=== RANKING DE PREÇOS ===  
1º Lugar - Fort: R$ 45.90  
2º Lugar - Giassi: R$ 48.50  
3º Lugar - Bistek (INCOMPLETA): R$ 32.00  
  
=== DETALHES DA MELHOR CESTA ===  
Supermercado: Fort (Total: R$ 45.90)  
- Arroz Branco 5kg (Urbano): R$ 39.90  
- Leite UHT Integral 1L (Tirol): R$ 6.00
```